

Unit-4: Inter-Process Communication and Synchronization Examples

Main Topic	Subtopics	What to Study
Inter-Process Communication (IPC)	Definition, need of IPC, process cooperation	Basic concept
Types of IPC	Shared memory, Message passing	Communication methods
Message Passing	Send and receive operations, blocking and non-blocking communication	Process communication
Strict Alternation	Algorithm, working, limitations	Software synchronization
Peterson's Solution	Algorithm, shared variables (flag, turn), working	Mutual exclusion
Producer-Consumer Problem	Buffer concept, semaphore solution	Process synchronization example
Dining Philosophers Problem	Resource sharing, deadlock problem	Synchronization example
Direct Communication	Process-to-process communication	Addressing processes directly
Indirect Communication	Communication via mailbox	Message queues
Pipes	Concept, working, types of pipes	Data transfer between processes
Signals	Definition, working	Event notification mechanism

Inter-Process Communication (IPC)

1. Definition: Inter-Process Communication (IPC) is a mechanism that allows **processes to communicate with each other and exchange data** while executing in an operating system.

Processes running on a system often need to **share information, coordinate actions, and synchronize their execution**. IPC provides the methods that enable this communication.

Simple exam definition : Inter-Process Communication is a method used by processes to exchange information and synchronize their activities in a computer system.

Need for Inter-Process Communication

Processes in an operating system may need to work together. IPC is required when:

Reason	Explanation
Information sharing	Processes share data
Computation speedup	Tasks are divided among processes
Modularity	System divided into smaller processes
Convenience	Multiple tasks performed simultaneously

Example:

A **web browser process** may have multiple processes for:

- rendering web pages
- downloading files
- handling user input

These processes communicate using IPC.

Types of Inter-Process Communication

There are two main IPC models.

IPC Type	Description
Shared Memory	Processes share a common memory space

Message Passing	Processes exchange messages
------------------------	-----------------------------

1. Shared Memory

Concept In the **shared memory model**, processes communicate by sharing a common region of memory.

Steps:

1. The operating system creates a shared memory region.
2. Multiple processes access this region.
3. Processes read and write data in shared memory.

Example:

Shared Memory Area

Process P1 → write data

Process P2 → read data

Advantages

Advantage	Explanation
Fast communication	No message copying
Efficient data sharing	Direct memory access
Low overhead	Minimal system call usage

Disadvantages

Disadvantage	Explanation
Synchronization required	Must control concurrent access
Possible race conditions	Multiple processes may modify data

2. Message Passing

Concept : In the **message passing model**, processes communicate by sending and receiving messages.

Communication occurs using system calls such as:

- send(message)
- receive(message)

Example:

Process P1 → send(message) → Process P2

Process P2 → receive(message)

Advantages

Advantage	Explanation
Simple implementation	Easier to manage
No shared memory conflicts	Safe communication
Better security	Processes remain isolated

Disadvantages

Disadvantage	Explanation
Slower communication	Message copying required
Higher system overhead	Multiple system calls

IPC Communication Diagram

Process P1



Inter-Process Communication



Process P2

Communication may occur through:

- shared memory
- message passing
- pipes
- signals

Real-World Examples of IPC

System	Example
Operating system services	Background processes communicating
Web browser	Multiple tabs exchanging data
Database systems	Multiple users accessing records
Network servers	Client-server communication

Advantages of IPC

Advantage	Explanation
Improves system performance	Multiple processes work together
Supports multitasking	Processes cooperate efficiently
Enables modular design	Programs divided into smaller processes

Disadvantages of IPC

Disadvantage	Explanation
Complex synchronization	Requires coordination
Communication overhead	Data transfer cost
Possible deadlocks	Processes waiting indefinitely

Key Points for Exams

- IPC allows **process communication and cooperation**
- Two major models:
 - Shared memory
 - Message passing
- Used in **multitasking operating systems**

Short Conclusion

Inter-Process Communication (IPC) enables processes running in an operating system to exchange information and coordinate their execution. It is essential in multitasking systems where multiple processes must cooperate to perform complex tasks efficiently.

Types of Inter-Process Communication (IPC)

1. Definition : Types of IPC refer to the different methods used by processes to **exchange information and coordinate execution** in an operating system.

IPC mechanisms enable processes to communicate either by **sharing memory** or by **sending messages**.

Simple exam definition : IPC can be implemented using two main methods: **Shared Memory** and **Message Passing**.

Two Main Types of IPC

IPC Method	Description
Shared Memory	Processes communicate by sharing a common memory region
Message Passing	Processes exchange information through messages

1. Shared Memory

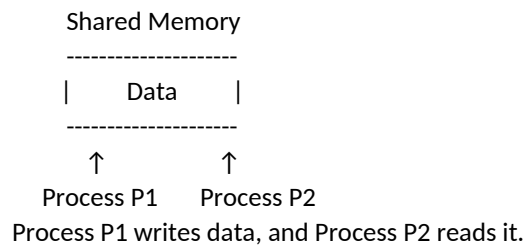
Concept : In the **shared memory model**, multiple processes access a **common memory area** created by the operating system.

Processes read and write data in this shared region to communicate.

Steps:

1. OS creates a shared memory segment.
2. Processes attach to this shared memory.
3. Processes exchange data through the shared memory region.

Shared Memory Diagram



Example

Suppose two processes share a variable:

shared_value = 10

Process P1 updates the value, and Process P2 reads the updated value.

Advantages of Shared Memory

Advantage	Explanation
Fast communication	Direct memory access
Low overhead	Fewer system calls
Efficient data transfer	Large data can be shared easily

Disadvantages of Shared Memory

Disadvantage	Explanation
Synchronization required	Processes must coordinate access
Possible race conditions	Concurrent access may cause errors
Security concerns	Shared data may be modified improperly

2. Message Passing

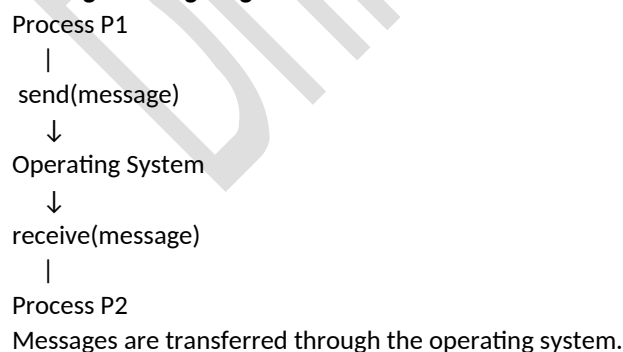
Concept : In the **message passing model**, processes communicate by **sending and receiving messages**.

The operating system manages the communication between processes.

Communication operations:

Operation	Purpose
send(message)	Sends message to another process
receive(message)	Receives message from another process

Message Passing Diagram



Example

Process P1 sends data to Process P2:

send(data)

receive(data)

Process P2 receives and processes the message.

Advantages of Message Passing

Advantage	Explanation
Safe communication	No shared memory conflicts
Simpler synchronization	OS controls message exchange
Suitable for distributed systems	Processes may run on different machines

Disadvantages of Message Passing

Disadvantage	Explanation
Slower communication	Data copying required
Higher overhead	Multiple system calls
Limited performance	Large data transfer is costly

Comparison of IPC Methods

Feature	Shared Memory	Message Passing
Communication method	Shared memory region	Message exchange
Speed	Faster	Slower
Synchronization	Required	Handled by OS
Complexity	More complex	Simpler
Use cases	High-speed communication	Distributed systems

Key Points for Exams

- IPC enables communication between processes
- Two main IPC types:
 - **Shared Memory**
 - **Message Passing**
- Shared memory is **faster**
- Message passing is **simpler and safer**

Short Conclusion

Inter-Process Communication can be implemented using shared memory or message passing. Shared memory allows processes to communicate by accessing a common memory space, while message passing enables processes to exchange information through messages managed by the operating system.

Message Passing

1. Definition : **Message Passing** is an Inter-Process Communication (IPC) method in which processes communicate with each other by **sending and receiving messages**.

In this method, processes do not share memory. Instead, the operating system provides communication mechanisms that allow processes to exchange information safely.

Simple exam definition : Message passing is a communication technique in which processes exchange data by sending and receiving messages through the operating system.

Basic Idea of Message Passing

In message passing communication:

- One process **sends a message**
- Another process **receives the message**

The operating system manages the communication between processes.

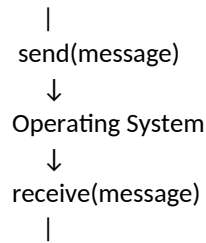
Basic operations:

Operation	Purpose
send(message)	Sends a message to another process

receive(message)	Receives a message from another process
------------------	---

Message Passing Communication Model

Process P1



Process P2

The operating system acts as a **mediator between processes**.

Components of Message Passing

Component	Description
Sender	Process that sends the message
Receiver	Process that receives the message
Message	Data being transferred
Communication link	Channel through which communication occurs

Types of Message Passing Communication

Message passing can be categorized based on how communication occurs.

Type	Description
Direct Communication	Processes communicate directly
Indirect Communication	Processes communicate through mailbox or message queue

1. Direct Communication

In direct communication, the sender and receiver **explicitly name each other**.

Example:

send(P2, message)

receive(P1, message)

Here:

- Process **P1 sends message to P2**
- Process **P2 receives message from P1**

Characteristics

Feature	Description
Explicit naming	Processes must know each other
Communication link	Established automatically
One-to-one communication	Only between two processes

2. Indirect Communication

In indirect communication, processes communicate using a **mailbox or message queue**.

Example:

send(mailbox, message)

receive(mailbox, message)

Processes do not communicate directly; they use a shared mailbox.

Characteristics

Feature	Description
Mailbox based communication	Messages stored in mailbox
Multiple processes	Many processes can communicate
Flexible communication	Processes need not know each other

Blocking and Non-Blocking Communication

Message passing may operate in two modes.

Mode	Description
Blocking Communication	Sender waits until message is received
Non-Blocking Communication	Sender continues execution after sending message

1. Blocking Communication

In blocking communication:

- The sending process waits until the message is delivered.
- The receiving process waits until a message arrives.

Example:

send(message) → wait until received

receive(message) → wait until message arrives

2. Non-Blocking Communication

In non-blocking communication:

- The sending process sends a message and continues execution.
- The receiving process checks if a message is available.

Example:

send(message) → continue execution

receive(message) → check mailbox

Advantages of Message Passing

Advantage	Explanation
No shared memory conflicts	Processes remain isolated
Simpler synchronization	OS manages message exchange
Secure communication	Processes cannot directly modify memory

Disadvantages of Message Passing

Disadvantage	Explanation
Slower communication	Messages must be copied
Higher system overhead	Multiple system calls required
Limited data transfer speed	Large data transfers are costly

Real-World Examples

System	Example
Client-server systems	Web server communicating with client
Network communication	Data exchanged between computers
Distributed systems	Processes communicating over network

Key Points for Exams

- Message passing is an **IPC mechanism**
- Processes communicate using **send() and receive()**
- Communication types:
 - Direct communication
 - Indirect communication
- Modes:
 - Blocking
 - Non-blocking

Short Conclusion

Message passing is an inter-process communication method in which processes exchange information through messages without sharing memory. It provides safe and controlled communication between processes and is widely used in distributed systems and client-server architectures.

Strict Alternation

1. Definition: Strict Alternation is a simple software solution used to solve the **critical section problem** in process synchronization.

In this method, two processes are allowed to enter the critical section **alternately using a shared variable called turn**.

Simple exam definition : Strict alternation is a synchronization technique in which processes enter the critical section alternately based on a shared turn variable.

Basic Idea of Strict Alternation

The algorithm uses a shared variable:

Turn This variable indicates **which process is allowed to enter the critical section**.

Example:

Value of turn	Process Allowed
0	Process P0
1	Process P1

Processes take turns executing the critical section.

Algorithm for Strict Alternation

Process P0

```
while(turn != 0);  
critical section  
turn = 1;  
remainder section
```

Process P1

```
while(turn != 1);  
critical section  
turn = 0;  
remainder section
```

Working of Strict Alternation

Steps:

1. Initially turn = 0
2. Process **P0 enters the critical section**
3. After finishing, P0 sets turn = 1
4. Process **P1 now enters the critical section**
5. After finishing, P1 sets turn = 0

The processes continue to **alternate execution**.

Strict Alternation Diagram

turn = 0

Process P0 → Critical Section

↓

turn = 1

Process P1 → Critical Section

↓

turn = 0

Process P0 → Critical Section

Processes take turns accessing the critical section.

Example Scenario

Two processes share a printer.

Step	Execution
1	turn = 0 → P0 prints
2	P0 finishes and sets turn = 1
3	P1 prints
4	P1 sets turn = 0

This ensures only **one process prints at a time**.

Advantages of Strict Alternation

Advantage	Explanation
Ensures mutual exclusion	Only one process enters critical section
Simple implementation	Easy algorithm
No race condition	Shared resource protected

Disadvantages of Strict Alternation

Disadvantage	Explanation
Violates progress condition	A process may wait unnecessarily
Inefficient CPU usage	Busy waiting occurs
Works only for two processes	Not scalable

Main Problem of Strict Alternation

Strict alternation violates the **progress condition**.

Example:

- Process P0 finishes its critical section.
- P1 is not interested in entering.
- P0 must still wait because turn = 1.

Thus the CPU remains idle even though P0 wants to execute.

Key Points for Exams

- Strict alternation is a **software solution for synchronization**
- Uses shared variable **turn**
- Allows processes to enter critical section **alternately**
- Ensures **mutual exclusion**
- Does **not satisfy progress condition**

Short Conclusion

Strict alternation is a simple software-based synchronization technique that ensures mutual exclusion by allowing processes to enter the critical section alternately. However, it is inefficient because it violates the progress condition and causes unnecessary waiting.

Peterson's Solution

1. Definition : Peterson's Solution is a software-based algorithm used to solve the **critical section problem for two processes**.

It ensures that only one process enters the **critical section at a time**, preventing race conditions.

The algorithm uses two shared variables to coordinate access to the critical section.

Simple exam definition : Peterson's solution is a synchronization algorithm that ensures mutual exclusion between two processes using shared variables flag and turn.

Shared Variables Used

Peterson's algorithm uses two shared variables.

Variable	Purpose
flag[2]	Indicates whether a process wants to enter the critical section
turn	Indicates which process gets priority

Example:

```
boolean flag[2];
```

```
int turn;
```

Explanation:

Variable	Meaning
flag[0]	Process P0 wants to enter critical section
flag[1]	Process P1 wants to enter critical section
turn	Indicates which process gets priority

Peterson's Algorithm

Process P0

```
flag[0] = true;
```

```
turn = 1;
```

```
while(flag[1] && turn == 1);
```

```
critical section
```

```
flag[0] = false;
```

```
remainder section
```

Process P1

```
flag[1] = true;
```

```
turn = 0;
```

```
while(flag[0] && turn == 0);
```

```
critical section
```

```
flag[1] = false;
```

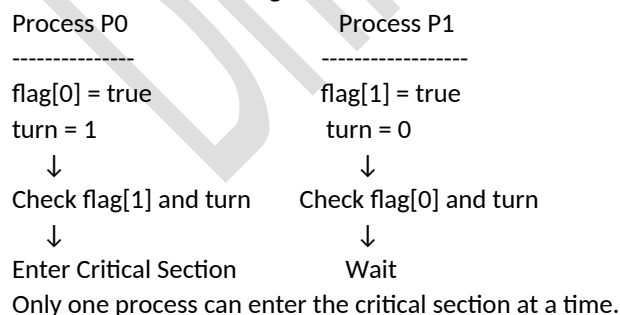
```
remainder section
```

Working of Peterson's Solution

Steps:

1. A process sets its flag to **true**, indicating that it wants to enter the critical section.
2. It sets the **turn variable** to give priority to the other process.
3. If the other process also wants to enter the critical section, the **turn variable decides which process enters first**.
4. After finishing the critical section, the process resets its flag to **false**.

Peterson's Solution Diagram



Example Scenario

Two processes want to update a shared variable.

Initial value:

```
count = 10
```

Both processes request entry to the critical section.

Peterson's solution ensures:

- only one process modifies the variable
- the other process waits until the critical section becomes free.

Advantages of Peterson's Solution

Advantage	Explanation
Ensures mutual exclusion	Only one process enters critical section
Prevents race conditions	Shared data remains consistent
Satisfies progress condition	No unnecessary waiting
Bounded waiting guaranteed	Every process gets a chance

Disadvantages of Peterson's Solution

Disadvantage	Explanation
Works only for two processes	Not scalable to many processes
Busy waiting	CPU cycles wasted
Software-only solution	Not efficient for large systems

Conditions Satisfied by Peterson's Solution

Peterson's algorithm satisfies all three requirements of the critical section problem.

Condition	Status
Mutual Exclusion	Satisfied
Progress	Satisfied
Bounded Waiting	Satisfied

Key Points for Exams

- Peterson's solution is a **software synchronization algorithm**
- Works for **two processes**
- Uses shared variables:
 - flag[]
 - turn
- Ensures **mutual exclusion**

Short Conclusion

Peterson's solution is a classical synchronization algorithm that solves the critical section problem for two processes. It uses shared variables to coordinate access and guarantees mutual exclusion, progress, and bounded waiting, ensuring safe execution of concurrent processes.

Producer-Consumer Problem

1. Definition : The **Producer-Consumer Problem** is a classical synchronization problem in operating systems that describes coordination between processes that **produce data and processes that consume data**.

Both processes share a **common buffer** where produced items are stored temporarily before being consumed.

Simple exam definition : The producer-consumer problem is a synchronization problem where producer processes generate data and consumer processes use that data from a shared buffer.

Basic Concept

Two types of processes exist:

Process	Role
Producer	Produces data items
Consumer	Consumes data items

Both processes share a **buffer (queue)**.

Example Scenario

Example:

- Producer → generates data
- Consumer → reads and processes data

Real-world examples:

System	Example
Printing system	Program sends print jobs (producer), printer processes them (consumer)
Keyboard input	Keyboard produces characters, program reads them
Video streaming	Server produces frames, player consumes frames

Shared Buffer

The shared buffer is a memory area used to temporarily store data.

Producer → Buffer → Consumer

The producer places items in the buffer, and the consumer removes them.

Two Main Problems

Two issues may occur in the producer-consumer system.

Problem	Explanation
Buffer Overflow	Producer tries to add item when buffer is full
Buffer Underflow	Consumer tries to remove item when buffer is empty

These problems require **synchronization mechanisms**.

Semaphore Solution

To solve the problem, three semaphores are used.

Semaphore	Purpose
mutex	Ensures mutual exclusion
empty	Counts empty buffer slots
full	Counts filled buffer slots

Initial values:

mutex = 1

empty = n (buffer size)

full = 0

Producer Algorithm

wait(empty)

wait(mutex)

insert item into buffer

signal(mutex)

signal(full)

Explanation:

Step	Meaning
wait(empty)	Wait if buffer is full
wait(mutex)	Enter critical section
insert item	Add item to buffer
signal(mutex)	Leave critical section
signal(full)	Increase filled slots

Consumer Algorithm

wait(full)

wait(mutex)

remove item from buffer

signal(mutex)

signal(empty)

Explanation:

Step	Meaning
wait(full)	Wait if buffer is empty
wait(mutex)	Enter critical section
remove item	Consume item
signal(mutex)	Leave critical section
signal(empty)	Increase empty slots

Producer-Consumer Diagram

Producer



Produce Item



Shared Buffer



Consumer

The semaphore mechanism ensures that **buffer overflow and underflow do not occur**.

Advantages of Producer-Consumer Model

Advantage	Explanation
Efficient resource use	Producer and consumer work concurrently
Better performance	Tasks executed in parallel
Supports multitasking	Multiple producers and consumers possible

Disadvantages

Disadvantage	Explanation
Complex synchronization	Requires semaphore management
Deadlock risk	Improper semaphore use may cause deadlock
Programming difficulty	Harder to debug concurrency errors

Key Points for Exams

- Classical synchronization problem
- Two processes:
 - producer
 - consumer
- Shared buffer used for data exchange
- Solved using **semaphores**
- Prevents **buffer overflow and underflow**

Short Conclusion

The producer-consumer problem illustrates how synchronization is required when processes share a common buffer. By using semaphores, the system ensures that producers do not insert items into a full buffer and consumers do not remove items from an empty buffer, maintaining correct and efficient process coordination.

Dining Philosophers Problem

1. Definition : The **Dining Philosophers Problem** is a classical synchronization problem used to illustrate issues of **resource sharing, deadlock, and process synchronization** in operating systems.

In this problem, several philosophers sit around a circular table and share limited resources (chopsticks). Each philosopher must acquire the required resources before performing an action.

Simple exam definition : The Dining Philosophers Problem is a synchronization problem where multiple processes compete for limited resources, which may lead to deadlock or starvation.

Basic Concept

- Five philosophers sit around a circular table.
- Each philosopher alternates between **thinking and eating**.
- A philosopher needs **two chopsticks** (left and right) to eat.
- Chopsticks are placed between each pair of philosophers.

Setup of the Problem

Item	Description
Philosophers	Processes
Chopsticks	Shared resources
Eating	Using resources
Thinking	Idle state

Each philosopher must acquire **two chopsticks** before eating.

Dining Philosophers Arrangement



Each philosopher shares chopsticks with neighbors.

Working of the Problem

Each philosopher performs the following steps:

1. Think
2. Pick up left chopstick
3. Pick up right chopstick
4. Eat
5. Put down chopsticks
6. Continue thinking

Problem Scenario

If all philosophers pick up their **left chopstick simultaneously**, each philosopher waits for the right chopstick.

Example:

Philosopher	Action
P1	picks left chopstick
P2	picks left chopstick
P3	picks left chopstick
P4	picks left chopstick
P5	picks left chopstick

Now every philosopher is waiting for the other chopstick.

This results in **deadlock**.

Problems Illustrated by Dining Philosophers

Problem	Explanation
---------	-------------

Deadlock	All processes wait indefinitely
Starvation	Some philosophers never get resources
Resource contention	Multiple processes compete for resources

Possible Solutions

Several synchronization techniques can solve the problem.

Method	Explanation
Semaphore solution	Control access to chopsticks
Resource hierarchy	Pick chopsticks in fixed order
Allow only four philosophers	Prevent circular waiting
Monitor solution	High-level synchronization

Semaphore Solution Idea

Each chopstick can be represented by a **binary semaphore**.

Algorithm outline:

```
wait(chopstick[i])
wait(chopstick[(i+1)%5])
```

eat

```
signal(chopstick[i])
signal(chopstick[(i+1)%5])
```

This ensures a philosopher acquires both chopsticks before eating.

Advantages of Dining Philosophers Model

Advantage	Explanation
Illustrates synchronization issues	Demonstrates resource sharing
Shows deadlock conditions	Useful teaching example
Improves understanding of concurrency	Helps design safer algorithms

Key Points for Exams

- Classical synchronization problem
- Five philosophers share five chopsticks
- Demonstrates:
 - deadlock
 - starvation
 - resource contention
- Solved using semaphores or monitors

Short Conclusion

The Dining Philosophers Problem demonstrates the challenges of synchronizing processes that share limited resources. Without proper synchronization mechanisms, processes may experience deadlock or starvation. The problem helps illustrate how operating systems manage resource allocation safely.

Direct Communication

1. Definition : Direct Communication is a message passing method in Inter-Process Communication (IPC) where the **sender and receiver processes explicitly name each other** while sending and receiving messages. In this communication model, processes communicate **directly without using any intermediate mailbox or message queue**.

Simple exam definition : Direct communication is a message passing technique in which the sending process directly specifies the receiving process.

Basic Concept

In direct communication, two processes establish a **direct communication link**.

Example:

send(P2, message)

receive(P1, message)

Explanation:

Operation	Meaning
send(P2, message)	Process P1 sends message to P2
receive(P1, message)	Process P2 receives message from P1

Direct Communication Model

Process P1

|

send(message)

↓

Process P2

The processes communicate **directly with each other**.

Characteristics of Direct Communication

Feature	Description
Explicit naming	Sender and receiver must know each other
Communication link	Automatically established between processes
One-to-one communication	Only two processes communicate
Bidirectional communication	Messages can flow both ways

Properties of Direct Communication

Property	Explanation
Automatic link establishment	Link created when processes communicate
Unique communication link	Only one link exists between two processes
Direct addressing	Processes directly identify each other

Example of Direct Communication

Suppose two processes communicate:

Process	Action
Process P1	Sends data
Process P2	Receives data

Example:

P1 → send(P2, data)

P2 → receive(P1, data)

Real-World Example

System	Example
Client-server model	Client directly sends request to server
Process communication	One process sending data directly to another

Advantages of Direct Communication

Advantage	Explanation
Simple implementation	Easy communication model
Fast communication	Direct message transfer
Clear communication path	Sender and receiver known

Disadvantages of Direct Communication

Disadvantage	Explanation
Limited flexibility	Processes must know each other
Difficult for large systems	Not suitable for many processes

Tight coupling

Processes strongly dependent

Direct Communication Diagram

Process P1

|
send(message)
↓

Process P2

The message flows directly between processes.

Key Points for Exams

- Part of **message passing IPC**
- Processes **directly specify sender and receiver**
- Communication is **one-to-one**
- Link is automatically established

Short Conclusion

Direct communication is a message passing technique where processes explicitly name each other while exchanging messages. It establishes a direct link between two processes, enabling simple and efficient communication, though it may lack flexibility in larger systems.

Indirect Communication

1. Definition : Indirect Communication is a message passing method in Inter-Process Communication (IPC) where processes communicate through an **intermediate mailbox (or message queue)** instead of communicating directly with each other.

In this method, the sender and receiver **do not need to know each other's identity**. Messages are sent to and received from a mailbox.

Simple exam definition : Indirect communication is a message passing technique in which processes communicate through a shared mailbox or message queue.

Basic Concept : In indirect communication, processes use a **mailbox** to exchange messages.

Example operations:

send(mailbox, message)

receive(mailbox, message)

Explanation:

Operation	Meaning
send(mailbox, message)	Send message to mailbox
receive(mailbox, message)	Receive message from mailbox

Indirect Communication Model

Process P1
|
send(message)
↓
Mailbox
↓
receive(message)
|
Process P2

The mailbox acts as an **intermediate storage for messages**.

Mailbox Concept

A **mailbox** is a logical object used to store messages temporarily.

Characteristics:

Feature	Description
Stores messages	Messages remain until received
Shared access	Multiple processes may use the mailbox
Managed by OS	Operating system manages mailbox

Communication Links in Indirect Communication

Link Type	Description
One-to-one	One sender and one receiver
Many-to-one	Multiple senders, single receiver
One-to-many	One sender, multiple receivers
Many-to-many	Multiple senders and receivers

This flexibility makes indirect communication suitable for **large systems**.

Example of Indirect Communication

Suppose two processes communicate using a mailbox.

P1 → send(mailbox1, data)

P2 → receive(mailbox1, data)

The processes communicate **through the mailbox** rather than directly.

Real-World Examples

System	Example
Email system	Messages stored in mailbox
Message queues	Applications exchanging data
Distributed systems	Processes communicating over network

Advantages of Indirect Communication

Advantage	Explanation
Flexible communication	Processes do not need to know each other
Supports many processes	Multiple senders and receivers
Better modular design	Loosely coupled processes

Disadvantages of Indirect Communication

Disadvantage	Explanation
Message management overhead	OS must manage mailbox
Possible message delay	Messages may wait in mailbox
Additional storage required	Mailbox must store messages

Direct vs Indirect Communication

Feature	Direct Communication	Indirect Communication
Addressing	Sender specifies receiver	Communication via mailbox
Flexibility	Less flexible	More flexible
Communication link	One-to-one	Many-to-many possible
Process dependency	Strong coupling	Loosely coupled

Key Points for Exams

- Indirect communication uses a **mailbox or message queue**
- Processes communicate **without knowing each other**
- Supports **multiple senders and receivers**
- More flexible than direct communication

Short Conclusion

Indirect communication is a message passing technique in which processes exchange messages through an intermediate mailbox. It provides flexible communication between multiple processes and reduces dependency between sender and receiver.

Pipes

1. Definition: A **Pipe** is a communication mechanism used in operating systems to allow **data transfer between processes**.

It is one of the simplest forms of **Inter-Process Communication (IPC)** where the output of one process becomes the input of another process.

Simple exam definition: A pipe is an IPC method that allows communication between processes using a data channel.

Basic Concept of Pipes

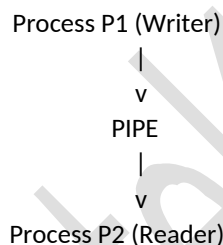
A pipe works like a **data channel** connecting two processes.

- One process **writes data into the pipe**
- Another process **reads data from the pipe**

Example:

Process P1 → writes data → Pipe → reads data → Process P2

Pipe Communication Model



The pipe transfers data from the **writer process to the reader process**.

Characteristics of Pipes

Feature	Description
Unidirectional communication	Data flows in one direction
Temporary communication	Exists while processes are running
Parent-child relationship	Usually used between related processes
Kernel managed	Operating system manages pipe buffer

Types of Pipes

There are two types of pipes used in operating systems.

Type	Description
Ordinary Pipe (Unnamed Pipe)	Used between related processes
Named Pipe (FIFO)	Used between unrelated processes

1. Ordinary Pipes

Concept : Ordinary pipes are used for communication between **processes with a parent-child relationship**.

These pipes are created by the operating system and exist only during process execution.

Example: Parent Process → Pipe → Child Process

Characteristics

Feature	Description
Temporary	Exists only while processes are running
Unidirectional	Data flows in one direction
Related processes	Parent-child processes only

2. Named Pipes (FIFO)

Concept : Named pipes allow communication between **unrelated processes**.

They are called **FIFO (First In First Out)** because data is read in the same order it was written.

Example: Process A → Named Pipe → Process B

Characteristics

Feature	Description
Persistent	Exists in the file system
Bidirectional communication	Possible using two pipes
Unrelated processes	Processes need not be related

Example of Pipe Usage

Example in command-line systems: command1 | command2

Example: ls | sort

Explanation:

Command	Action
ls	Lists files
sort	Sorts the list

The output of **ls** becomes input to **sort**.

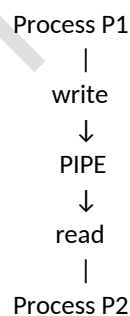
Advantages of Pipes

Advantage	Explanation
Simple IPC mechanism	Easy to implement
Efficient data transfer	Direct communication
Useful in process chaining	Connects multiple commands

Disadvantages of Pipes

Disadvantage	Explanation
Limited communication	Mostly unidirectional
Process dependency	Often requires related processes
Temporary nature	Ordinary pipes disappear after process ends

Simple Pipe Diagram



Key Points for Exams

- Pipes are used for **inter-process communication**
- Data flows from **writer to reader**
- Two types:
 - Ordinary pipes
 - Named pipes
- Often used in **Unix/Linux command pipelines**

Short Conclusion

Pipes are a simple IPC mechanism that allows one process to send data to another process through a communication channel. They are widely used in operating systems to connect processes and support efficient data transfer between programs.

Signals

1. Definition : A **Signal** is a mechanism used in operating systems to **notify a process that a specific event has occurred**.

Signals are used for **simple communication between processes or between the operating system and processes**.

Simple exam definition : A signal is a notification sent to a process to inform it that an event has occurred.

Basic Concept of Signals

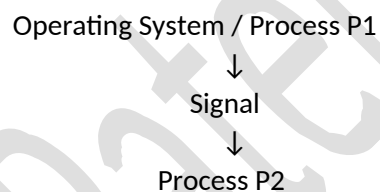
Signals are used to **interrupt the normal execution of a process** and inform it about certain events.

Examples of events:

- process termination
- illegal memory access
- keyboard interrupt
- timer expiration

When a signal is received, the process can **handle the signal or terminate**.

Signal Communication Model



A signal is sent to notify the process about an event.

How Signals Work

Steps:

1. An event occurs.
2. The operating system generates a signal.
3. The signal is sent to the target process.
4. The process performs an action in response to the signal.

Signal Handling

When a process receives a signal, it can respond in three ways.

Response	Description
Default action	OS performs default operation
Ignore signal	Process ignores the signal
Custom handler	Process defines its own response

Common Signals in Operating Systems

Signal	Meaning
SIGINT	Interrupt signal (Ctrl + C)
SIGTERM	Termination request
SIGKILL	Forcefully terminate process
SIGSTOP	Stop process execution

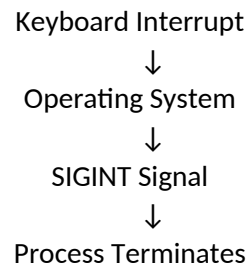
SIGALRM	Alarm signal (timer event)
----------------	----------------------------

Example of Signal Usage

Example scenario:

- User presses **Ctrl + C**
- Operating system sends **SIGINT**
- The running process stops execution

Example flow:



Real-World Example

System	Example
User interrupt	Pressing Ctrl+C to stop program
Process termination	OS stopping a process
Timer events	Alarm signals
Error handling	Illegal instruction detection

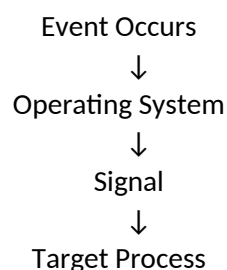
Advantages of Signals

Advantage	Explanation
Simple communication	Easy notification mechanism
Quick event handling	Immediate response
Low overhead	Minimal system resources used

Disadvantages of Signals

Disadvantage	Explanation
Limited data transfer	Only signal notification
Complex handling	Difficult to manage many signals
Possible signal loss	Multiple signals may overlap

Signal Diagram



The signal informs the process about the event.

Key Points for Exams

- Signals are used for **process notification**
- Sent by **operating system or other processes**

- Used for **interrupts and event handling**
- Examples: **SIGINT, SIGTERM, SIGKILL**

Short Conclusion

Signals are a simple IPC mechanism used by operating systems to notify processes about events such as interrupts, errors, or termination requests. They enable processes to respond quickly to system events and maintain proper execution control.

Event Counters

1. Introduction: An Event Counter is a synchronization mechanism used in operating systems to coordinate processes based on the occurrence of events.

It helps processes wait for specific events and continue execution only after those events occur.

Event counters are mainly used in concurrent systems for process synchronization.

Definition: An Event Counter is a synchronization variable used to count and track events occurring in a system.

Simple Exam Definition: An event counter is a synchronization mechanism that allows processes to wait for and signal events.

Basic Idea

Processes often depend on certain events.

Example:

- A process waits until data becomes available
- Another process produces the data

Event counters coordinate this execution.

Real-Life Example

Suppose:

- Students wait for exam results
- Results are announced later

The announcement acts like an event.

Students continue only after event occurs.

Need for Event Counters

Event counters are needed for:

- Process synchronization
- Event tracking
- Coordinating concurrent execution
- Preventing race conditions

Working Principle

An event counter keeps track of event occurrences.

When:

- event occurs → counter increases
- process waits → checks counter value

Basic Operations

Operation	Purpose
Advance()	Increment event counter

Await()	Wait until event occurs
Read()	Read counter value

Event Counter Model

Process P1 → Generates Event

|
Event Counter

|
Process P2 waits until event occurs

Example Scenario

Suppose:

- Producer creates item
- Consumer waits for item

When producer finishes:

- event counter increases
- consumer proceeds

Event Counter Diagram

Event Occurs
|
Advance Counter
|
Waiting Process Continues

Event Counter Variables

Two main values are used:

Variable	Meaning
Event Count	Number of completed events
Waiting Condition	Event required by process

Simple Example

Initial:

event_count = 0

Producer completes task:

event_count = 1

Consumer waiting for:

event_count >= 1

Now consumer continues.

Synchronization Using Event Counters

Event counters help:

- coordinate processes
- control execution order
- avoid busy waiting

Event Counter Operations

1. Advance Operation

Used when event occurs.

Example:

Advance(event)

Counter increases.

2. Await Operation

Process waits until event reaches required value.

Example:

Await(event)

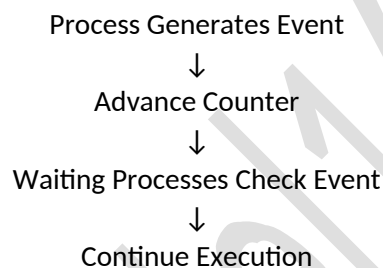
3. Read Operation

Returns current event count.

Example:

Read(event)

Event Counter Workflow



Advantages of Event Counters

Advantage	Explanation
Efficient synchronization	Coordinates processes
Reduces busy waiting	Better CPU usage
Ordered execution	Controls event sequence
Useful in concurrent systems	Process coordination

Disadvantages of Event Counters

Disadvantage	Explanation
Complex implementation	Harder than simple locks
Limited usage	Specialized synchronization
Extra management	Event tracking needed

Event Counters vs Semaphores

Feature	Event Counter	Semaphore
Purpose	Event synchronization	Resource control
Main use	Track events	Mutual exclusion
Operation style	Event-based	Counter-based

Event Counters vs Signals

Feature	Event Counter	Signal
Communication	Synchronization	Notification
Persistence	Counter maintained	Temporary signal
Event tracking	Yes	No

Applications of Event Counters

Application	Usage
Producer-consumer systems	Event coordination
Concurrent programming	Synchronization
Parallel processing	Process coordination
Operating systems	Event management

Characteristics of Event Counters

Characteristic	Explanation
Event tracking	Counts occurrences
Synchronization support	Coordinates processes
Concurrent execution support	Useful in multitasking
Ordered process execution	Maintains sequence

Key Terms

Term	Meaning
Event	Occurrence in system
Counter	Tracks events
Advance()	Increase count
Await()	Wait for event
Synchronization	Coordinated execution

Important Exam Points

- Event counter is synchronization mechanism
- Tracks occurrence of events
- Used for process coordination
- Main operations:
 - Advance()
 - Await()
 - Read()
- Helps avoid busy waiting

Q4. Difference between semaphore and event counter?

Semaphore	Event Counter
Resource synchronization	Event synchronization

Short Conclusion

Event counters are synchronization mechanisms used in operating systems to coordinate concurrent processes based on event occurrences. They help maintain ordered execution, reduce busy waiting, and improve synchronization efficiency in concurrent systems.

Reader-Writer Problem

1. Introduction

The Reader-Writer Problem is a classical synchronization problem in operating systems that deals with processes accessing shared data simultaneously.

Two types of processes exist:

- Readers
- Writers

The challenge is to allow multiple readers to access data together while ensuring writers get exclusive access.

Definition: The Reader-Writer Problem is a synchronization problem where multiple readers can read shared data simultaneously, but writers require exclusive access to modify data.

Simple Exam Definition: The Reader-Writer Problem is a synchronization problem that manages simultaneous access to shared data by readers and writers.

Basic Concept

Process Type	Function
Reader	Reads shared data
Writer	Modifies shared data

Main Rule

Multiple Readers Allowed

Only One Writer Allowed

Why Writer Needs Exclusive Access

If:

- one writer modifies data
- readers read simultaneously

Then inconsistent or incorrect data may occur.

Real-Life Example

Library Example

User	Action
Students	Read books
Librarian	Updates books

Many students can read simultaneously.

But librarian updates records alone.

Problem Conditions

Condition	Requirement
Multiple readers	Allowed together
Multiple writers	Not allowed
Reader + writer together	Not allowed

Reader-Writer Model

Readers ---> Shared Data <--- Writers

Types of Reader-Writer Problems

There are mainly three versions.

Type	Priority
First Reader-Writer Problem	Reader priority
Second Reader-Writer Problem	Writer priority
Third Reader-Writer Problem	Fair scheduling

1. First Reader-Writer Problem

Readers get priority.

If readers continuously arrive:

- writers may starve

2. Second Reader-Writer Problem

Writers get priority.

Readers may wait longer.

3. Third Reader-Writer Problem

Fair scheduling between readers and writers.

Avoids starvation.

Synchronization Requirements

To solve Reader-Writer Problem:

- Mutual exclusion required
- Semaphores commonly used

Semaphore Variables Used

Semaphore	Purpose
mutex	Protect reader count
wrt	Control writer access

Shared Variable

readcount = 0

Tracks active readers.

Reader Algorithm

wait(mutex)

readcount++

if(readcount == 1)

 wait(wrt)

signal(mutex)

READ DATA

wait(mutex)

readcount--

if(readcount == 0)

 signal(wrt)

signal(mutex)

Reader Algorithm Explanation

Step	Purpose
wait(mutex)	Protect readcount
readcount++	Reader enters
first reader waits(wrt)	Block writers
read data	Access shared resource
last reader signal(wrt)	Allow writers

Writer Algorithm

wait(wrt)

WRITE DATA

signal(wrt)

Writer Algorithm Explanation

Step	Purpose
wait(wrt)	Get exclusive access
write data	Modify resource
signal(wrt)	Release resource

Working of Reader-Writer Problem

Scenario 1: Multiple Readers

Reader1 → Read

Reader2 → Read

Reader3 → Read

All allowed simultaneously.

Scenario 2: Writer Arrives

Writer waits until all readers finish.

Scenario 3: Writer Writing

No reader or writer allowed simultaneously.

Reader-Writer Diagram

Readers ---> Shared Database <--- Writer

Reader Priority Problem

If readers continuously enter:

Writer may wait forever.

This causes:

Starvation

Writer Priority Solution

Writers are given preference.

Prevents writer starvation.

Fair Solution

Fair scheduling ensures:

- no starvation
- balanced execution

Advantages of Reader-Writer Solution

Advantage	Explanation
Efficient reading	Multiple readers allowed
Data consistency	Writers exclusive
Improved concurrency	Better resource usage
Synchronization support	Controlled access

Disadvantages

Disadvantage	Explanation
Complex synchronization	Hard implementation
Starvation possible	Reader/writer starvation
Semaphore overhead	Synchronization cost

Applications of Reader-Writer Problem

Application	Usage
Database systems	Multiple users reading
File systems	Shared file access
Web servers	Concurrent client access
Operating systems	Shared resource management

Reader-Writer vs Producer-Consumer

Feature	Reader-Writer	Producer-Consumer
Shared access	Readers/Writers	Producers/Consumers
Main issue	Simultaneous reading	Buffer management
Synchronization focus	Exclusive writing	Buffer synchronization

Key Terms

Term	Meaning
Reader	Reads data
Writer	Modifies data
readcount	Active readers
mutex	Protects shared variable
wrt	Controls writing

Important Exam Points

- Classical synchronization problem
- Multiple readers allowed simultaneously
- Writers need exclusive access
- Uses semaphores
- Reader priority may cause starvation

Short Conclusion

The Reader-Writer Problem is a classical synchronization problem that controls access to shared data among multiple readers and writers. It allows multiple readers to access data simultaneously while ensuring writers receive exclusive access for safe data modification. Semaphores and synchronization mechanisms are used to prevent race conditions, maintain consistency, and coordinate concurrent execution.